JavaScript Interview Preparation Guide (0–3 Years)

A **professional, interview-first** JavaScript handbook for frontend/full-stack interviews.

Each question includes:

- **Short Interview Answer** (what to say in 10–20 seconds)
- **Explanation** (clear + accurate)
- **Example Code**
- **Best Practices**

---

1) JavaScript Basics

Question: What is JavaScript and where does it run?

Short Interview Answer: JavaScript is a dynamic programming language primarily used for web development. It runs in browsers (via JS engines like V8) and on servers using environments like Node.js.

Explanation: In the browser, JavaScript can read/modify the DOM, handle events, and make network requests. In Node.js, JavaScript runs outside the browser and is used for servers, tooling, scripts, etc.

Example Code:

```
console.log('Runs in browser console and Node.js');
```

Best Practices:

- Know differences between browser APIs (DOM, fetch) and Node APIs (fs, process).

Question: What is the difference between JavaScript and ECMAScript?

Short Interview Answer: ECMAScript is the language specification; JavaScript is the implementation of that spec (plus host APIs provided by browser/Node).

Explanation: Features like `let/const`, classes, modules are ECMAScript features. DOM APIs are not ECMAScript—they're browser APIs.

Example Code:

```
// ECMAScript feature
const x = 10;

// Browser API (not ECMAScript)
document.querySelector('div');
```

Best Practices:

- Say "JavaScript = ECMAScript + host environment APIs."

---

2) Variables (var, let, const)

Question: Difference between `var`, `let`, and `const`?

Short Interview Answer: `var` is function-scoped and hoisted with `undefined`. `let/const` are block-scoped and hoisted but in the Temporal Dead Zone. `const` prevents reassignment.

Explanation: `var` can be redeclared and causes bugs in loops/closures. `let` is safer for changing values. `const` is best by default for references you don't reassign.

Example Code:

```
function demo() {
  if (true) {
    var a = 1;
    let b = 2;
    const c = 3;
  }
  console.log(a); // 1
  // console.log(b); // ReferenceError
  // console.log(c); // ReferenceError
}
```

Best Practices:

- Prefer `const`, then `let`.
- Avoid `var` in modern codebases.

Question: What does `const` actually mean?

Short Interview Answer: `const` prevents reassignment of the variable binding, not mutation of objects.

Explanation: Objects/arrays declared with `const` can still change their properties/elements.

Example Code:

```
const user = { name: 'Vishal' };
user.name = 'Chandora'; // allowed
// user = {}; // TypeError (reassignment)
```

Best Practices:

- Use `Object.freeze` for shallow immutability (still not deep).

---

3) Data Types

Question: What are JavaScript primitive types?

Short Interview Answer: Primitives: `string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`. Everything else is an object.

Explanation: Primitives are immutable values. Objects are reference types.

Example Code:

```
typeof 'a' // 'string'
typeof 1   // 'number'
typeof null // 'object' (historical bug)
```

Best Practices:

- Remember: `typeof null === 'object'` is a common interview trap.

Question: Difference between `null` and `undefined`?

Short Interview Answer: `undefined` means “not assigned / missing”; `null` is an intentional empty value.

Explanation: A function without return returns `undefined`. You set `null` when you intentionally clear a value.

Example Code:

```
let x;
console.log(x); // undefined
x = null;
console.log(x); // null
```

Best Practices:

- Pick one convention for “no value” in APIs; be consistent.

---

4) Type Coercion

Question: What is type coercion in JavaScript?

Short Interview Answer: Type coercion is JavaScript automatically converting values between types (implicit) or conversions you do explicitly.

Explanation: Common in `+` operator, comparisons, and boolean contexts.

Example Code:

```
'5' + 1 // '51'
'5' - 1 // 4
Boolean('') // false
Number('42') // 42
```

Best Practices:

- Prefer explicit conversion for clarity (`Number()` , `String()`).

Question: Difference between `==` and `===` ?

Short Interview Answer: `==` allows type coercion; `===` checks both type and value.

Explanation: `==` has many edge cases; `===` is safer.

Example Code:

```
0 == '0'   // true
0 === '0'  // false
null == undefined // true
null === undefined // false
```

Best Practices:

- Use `===` in almost all cases.

---

5) Operators

Question: What are truthy and falsy values?

Short Interview Answer: Falsy: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Everything else is truthy.

Explanation: Used in conditions and logical operators.

Example Code:

```
if ('0') console.log('truthy');
if (0) console.log('never runs');
```

Best Practices:

- Don't confuse `'0'` (truthy) with `0` (falsy).

Question: Difference between `||` and `??`?

Short Interview Answer: `||` uses falsy checks; `??` uses nullish checks (`null` / `undefined` only).

Explanation: `??` is safer for values like `0` or `''`.

Example Code:

```
const count = 0;
count || 10; // 10 (maybe wrong)
count ?? 10; // 0  (correct)
```

Best Practices:

- Use `??` for defaulting when 0/'' are valid values.

---

6) Functions

Question: Function declaration vs function expression?

Short Interview Answer: Declarations are hoisted with their body; expressions assign a function to a variable and follow variable hoisting rules.

Explanation: You can call a function declaration before its definition.

Example Code:

```
sayHi();
function sayHi() { console.log('hi'); }

// greet(); // ReferenceError/TypeError depending on let/var
const greet = function () { console.log('hello'); };
```

Best Practices:

- Use consistent style across a codebase.

Question: Arrow functions vs regular functions?

Short Interview Answer: Arrow functions don't have their own `this`, `arguments`, or `prototype`. Regular functions do.

Explanation: Arrow functions capture `this` from the surrounding lexical scope.

Example Code:

```
const obj = {
  name: 'A',
  regular() { return this.name; },
  arrow: () => this?.name
};
```

Best Practices:

- Use regular functions for object methods when you need `this`.

---

7) Scope

Question: What is scope in JavaScript?

Short Interview Answer: Scope determines where variables are accessible: global, function, and block scope.

Explanation: `let/const` are block-scoped; `var` is function-scoped.

Example Code:

```
{
  let x = 1;
}
// console.log(x); // ReferenceError
```

Best Practices:

- Keep scopes small to reduce bugs.

---

8) Hoisting

Question: What is hoisting?

Short Interview Answer: Hoisting is JavaScript moving declarations to the top of their scope during compilation, affecting when variables/functions can be referenced.

Explanation: `var` is hoisted and initialized to `undefined`. `let/const` are hoisted but not initialized (TDZ).

Example Code:

```
console.log(a); // undefined
var a = 10;

// console.log(b); // ReferenceError (TDZ)
let b = 20;
```

Best Practices:

- Declare variables at top of scope for readability (even though hoisting exists).

---

9) Closures

Question: What is a Closure in JavaScript?

Short Interview Answer: A closure is a function that remembers variables from its outer scope even after the outer function has finished executing.

Explanation: Closures allow functions to access variables from their lexical environment. They are commonly used for data privacy and function factories.

Example Code:

```
function outer() {
  let count = 0;

  return function inner() {
    count++;
    console.log(count);
  };
}

const counter = outer();

counter(); // 1
counter(); // 2
```

Best Practices:

- Use closures for encapsulation, but avoid accidental memory leaks by keeping references minimal.

---

10) Execution Context

Question: What is an execution context?

Short Interview Answer: An execution context is the environment where JS code runs, containing variables, scope chain, and `this`.

Explanation: There's a global execution context and a new function execution context per function call.

Example Code:

```
function a() { function b() { return 1; } return b(); }
a();
```

Best Practices:

- Understand scope chain + lexical environment to explain closures.

---

11) Call Stack

Question: What is the call stack?

Short Interview Answer: The call stack is a LIFO structure that tracks function calls. Each call pushes a frame; return pops it.

Explanation: Stack overflows happen with deep recursion.

Example Code:

```
function recurse() { return recurse(); }
// recurse(); // RangeError: Maximum call stack size exceeded
```

Best Practices:

- Avoid unbounded recursion; use iteration or tail-call strategies (JS engines don't guarantee TCO).

---

12) Event Loop

Question: What is the event loop?

Short Interview Answer: The event loop coordinates executing synchronous code, then processing tasks (macrotasks) and microtasks, keeping JS non-blocking.

Explanation: Microtasks (Promise callbacks) run before the next macrotask (setTimeout, IO callbacks).

Example Code:

```
console.log('A');
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('B');
// A, B, promise, timeout
```

Best Practices:

- Avoid blocking the main thread with heavy work.

---

13) Asynchronous JavaScript

Question: What are common async patterns in JS?

Short Interview Answer: Callbacks, Promises, and async/await.

Explanation: Promises avoid callback nesting and allow composition.

Example Code:

```
fetch('/api')
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Best Practices:

- Always handle errors (`catch` / try-catch with async/await).

---

14) Promises

Question: What is a Promise?

Short Interview Answer: A Promise represents a future result: pending → fulfilled/rejected. It enables async composition.

Explanation: Promises have `.then`, `.catch`, `.finally`. They can be chained.

Example Code:

```
const p = new Promise((resolve) => setTimeout(() => resolve(42), 100));
p.then(v => console.log(v));
```

Best Practices:

- Use `Promise.all` for parallel tasks; handle partial failures if needed.

Question: `Promise.all` vs `Promise.allSettled`?

Short Interview Answer: `Promise.all` rejects on first rejection; `allSettled` waits for all and returns statuses.

Explanation: Use `allSettled` when you need results even if some fail.

Example Code:

```
await Promise.allSettled([fetch('/a'), fetch('/b')]);
```

Best Practices:

- Don't over-parallelize huge lists; use concurrency limits.

---

15) Async / Await

Question: How does `async/await` work?

Short Interview Answer: `async` makes a function return a Promise; `await` pauses inside that function until the Promise resolves/rejects.

Explanation: It's syntactic sugar over Promise chains.

Example Code:

```
async function load() {
  try {
    const res = await fetch('/api');
    return await res.json();
  } catch (e) {
    console.error(e);
    return null;
  }
}
```

Best Practices:

- Use `try/catch` around awaited operations.

---

16) Objects

Question: How do you copy objects in JS?

Short Interview Answer: Shallow copy: spread `{...obj}` or `Object.assign`. Deep copy: structuredClone (best), or JSON (limited).

Explanation: Spread copies only one level. Deep cloning is needed for nested references.

Example Code:

```
const a = { x: 1, nested: { y: 2 } };
const b = { ...a };
b.nested.y = 99;
console.log(a.nested.y); // 99 (shallow copy)
```

Best Practices:

- Prefer `structuredClone` when available.

Question: What is destructuring?

Short Interview Answer: Destructuring extracts properties/elements into variables.

Explanation: It improves readability and reduces repetitive access.

Example Code:

```
const user = { name: 'Vishal', role: 'Frontend' };
const { name, role } = user;
```

Best Practices:

- Use defaults in destructuring for safer code.

---

17) Arrays

Question: Difference between `map`, `forEach`, `filter`, `reduce`?

Short Interview Answer:

- `map` transforms and returns new array
- `forEach` iterates (no return)
- `filter` selects subset
- `reduce` aggregates into a single value

Explanation: These are functional iteration helpers.

Example Code:

```
const nums = [1,2,3];
nums.map(n => n*2);     // [2,4,6]
nums.filter(n => n>1);  // [2,3]
nums.reduce((sum,n) => sum+n, 0); // 6
```

Best Practices:

- Prefer `map/filter/reduce` for clarity, but don't overuse reduce when simple loops are clearer.

Question: What is the difference between `slice` and `splice`?

Short Interview Answer: `slice` returns a new array without changing original; `splice` mutates array.

Explanation: Splice is for insert/remove in-place.

Example Code:

```
const a = [1,2,3];
a.slice(1); // [2,3]
a.splice(1,1); // removes 2, a becomes [1,3]
```

Best Practices:

- Avoid mutation in React state; prefer non-mutating methods.

---

18) Prototypes

Question: What is prototypal inheritance?

Short Interview Answer: Objects can inherit properties via the prototype chain. If a property isn't found on the object, JS looks up the prototype.

Explanation: Functions have a `prototype` property used when creating objects via `new`.

Example Code:

```
const obj = {};
console.log(obj.toString); // from Object.prototype
```

Best Practices:

- Prefer classes for readability, but understand prototype chain for debugging.

---

19) Classes

Question: Are classes in JS truly classes?

Short Interview Answer: They are syntax sugar over prototypal inheritance.

Explanation: Methods are placed on the prototype.

Example Code:

```
class User {
  constructor(name) { this.name = name; }
  greet() { return `Hi ${this.name}`; }
}
```

Best Practices:

- Use classes when they improve clarity; avoid over-OOP.

---

20) This Keyword

Question: How does `this` work in JavaScript?

Short Interview Answer: `this` depends on how a function is called: method call → object, regular call → global/undefined (strict), `new` → new instance, `call/apply/bind` set it explicitly. Arrow functions capture lexical `this`.

Explanation: It’s runtime binding, not lexical (except arrow).

Example Code:

```
const obj = {
  x: 10,
  getX() { return this.x; }
};
obj.getX(); // 10

const f = obj.getX;
// f(); // undefined in strict mode
```

Best Practices:

- Use arrow functions carefully in objects; prefer methods.

---

21) DOM Manipulation

Question: How do you select elements in the DOM?

Short Interview Answer: Use `querySelector/querySelectorAll` for CSS selectors, or `getElementById` for IDs.

Explanation: `querySelector` returns first match. `querySelectorAll` returns a static NodeList.

Example Code:

```
const btn = document.querySelector('button.primary');
const items = document.querySelectorAll('.item');
```

Best Practices:

- Cache selectors; avoid repeated expensive queries.

Question: `innerHTML` vs `textContent`?

Short Interview Answer: `innerHTML` parses HTML (XSS risk). `textContent` sets plain text safely.

Explanation: If you insert user input with `innerHTML`, you can introduce XSS.

Example Code:

```
el.textContent = userInput; // safe
// el.innerHTML = userInput; // unsafe
```

Best Practices:

- Use `textContent` unless you truly need HTML.

---

22) Event Handling

Question: What is event bubbling and capturing?

Short Interview Answer: Events travel down in capturing phase then up in bubbling phase. By default listeners run in bubbling.

Explanation: Event delegation uses bubbling to handle events efficiently.

Example Code:

```
document.querySelector('ul').addEventListener('click', (e) => {
  if (e.target.matches('li')) console.log('Clicked', e.target);
});
```

Best Practices:

- Use delegation for lists/tables with many items.

---

23) Browser Storage

Question: Difference between localStorage, sessionStorage, and cookies?

Short Interview Answer: localStorage persists until cleared; sessionStorage lasts per tab session; cookies are sent with HTTP requests and can be HttpOnly.

Explanation: Cookies are better for auth when HttpOnly+Secure; localStorage is vulnerable to XSS.

Example Code:

```
localStorage.setItem('theme','dark');
sessionStorage.setItem('step','2');
document.cookie = 'x=1; SameSite=Lax';
```

Best Practices:

- Don’t store sensitive tokens in localStorage.

---

24) Modules

Question: What are ES modules and how do you use them?

Short Interview Answer: ES modules use `import/export` and run in strict mode by default. In browsers you load with `type="module"`.

Explanation: Modules have their own scope and support static analysis.

Example Code:

```
// math.js
export const add = (a,b) => a+b;

// app.js
import { add } from './math.js';
console.log(add(1,2));
```

Best Practices:

- Prefer ESM in modern codebases.

---

25) Error Handling

Question: How do you handle errors in async code?

Short Interview Answer: Use `.catch()` for Promises or `try/catch` inside async functions.

Explanation: Unhandled rejections can crash Node apps or cause silent failures.

Example Code:

```
async function run() {
  try {
    await doWork();
  } catch (e) {
    console.error('Failed', e);
  }
}
```

Best Practices:

- Centralize error handling and show user-friendly messages.

---

26) Performance Best Practices

Question: What are common JS performance best practices?

Short Interview Answer: Avoid blocking the main thread, debounce/throttle events, minimize DOM writes, use requestAnimationFrame for animations, and split code.

Explanation: DOM updates are expensive. Long tasks cause jank.

Example Code:

```
function debounce(fn, ms) {
  let id;
  return (...args) => {
    clearTimeout(id);
    id = setTimeout(() => fn(...args), ms);
  };
}
```

Best Practices:

- Batch DOM reads/writes.
- Use Web Workers for heavy computation.

---

27) Common Interview Traps

Question: Why does `[] + []` give an empty string?

Short Interview Answer: Because `+` triggers coercion to primitives; arrays become `''`, so `'' + ''` is `''`.

Explanation: Arrays `toString()` to `''` when empty.

Example Code:

```
[] + [] // ''
[] + {} // '[object Object]'
```

Best Practices:

- Prefer explicit conversion; avoid clever coercion.

Question: What is the output and why?

Short Interview Answer: `typeof null` is `'object'` due to a historical JS bug.

Explanation: It’s standardized for web compatibility.

Example Code:

```
typeof null; // 'object'
```

Best Practices:

- Use `value === null` to check for null.

Question: What is the difference between `for...in` and `for...of`?

Short Interview Answer: `for...in` iterates enumerable keys (including inherited). `for...of` iterates values of an iterable (arrays, strings, maps).

Explanation: Using `for...in` on arrays is usually wrong.

Example Code:

```
for (const k in {a:1}) console.log(k);
for (const v of [1,2,3]) console.log(v);
```

Best Practices:

- Use `for...of` for arrays.

---

28) Rapid Revision Cheat Sheet

- Prefer `const`, then `let` (avoid `var`)
- `===` over `==`
- Primitives: string, number, boolean, null, undefined, bigint, symbol
- `typeof null` is `'object'` (trap)
- Closure = function remembers outer scope
- Event loop: microtasks (Promises) before macrotasks (setTimeout)
- `||` vs `??`: falsy vs nullish
- Shallow vs deep copy: spread is shallow
- Arrow functions: lexical `this`
- DOM: use `textContent` for safety
- Storage: avoid tokens in localStorage

Generated from markdown for sharing (PDF-friendly).